Generative AI and its Applications

UE23CS342BA9

Evolution of LLMs

Dr. UMA D
Professor
Department of Computer Science and Engineering

Why Evolution Was Necessary?

Early systems failed because:

- **Long-range dependencies:** Word meaning often depends on distant context, which rule-based and early statistical models cannot retain or reference effectively.
- **Context-dependent semantics:** Words can have multiple meanings depending on usage, for example “bank” as a financial institution or a riverbank, requiring global context understanding.
- **Poor scalability of rules:** Covering all grammatical and semantic variations requires an exponentially growing number of handcrafted rules, making systems difficult to extend and maintain.

Example:

- “Bank” - river vs money

Early-NLP: Rule-Based Systems

Early NLP systems relied on manually written linguistic rules designed by domain experts. They used grammar rules, dictionaries, and pattern matching, performing well only in controlled environments.

Limitations:

- Rigid and brittle behavior
- High manual effort to maintain rules
- Poor handling of ambiguity and variation
- Limited scalability

These drawbacks motivated data-driven approaches.

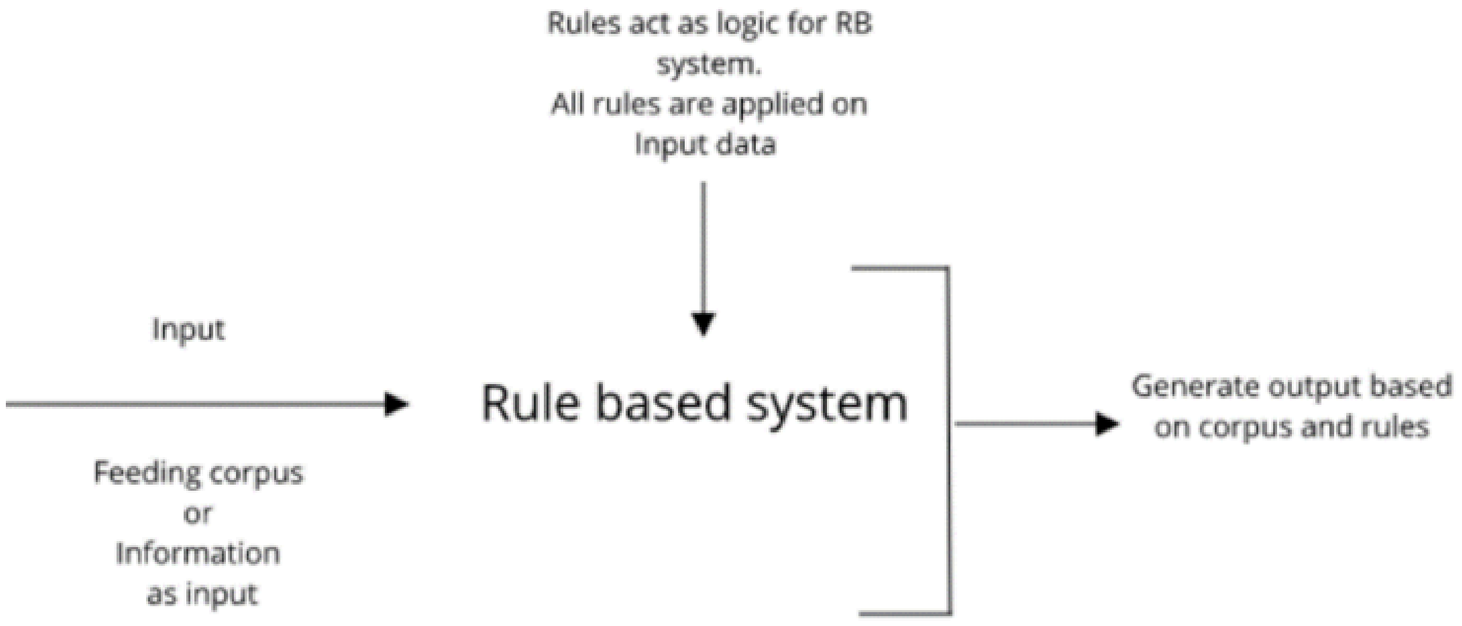


Image Source

Statistical Language Models (N-grams)

Statistical Language Models introduced probability into language processing. N-gram models predict the next word using the previous *n − 1* words, based on frequency counts from data.
They marked a shift from handcrafted rules to data-driven learning.

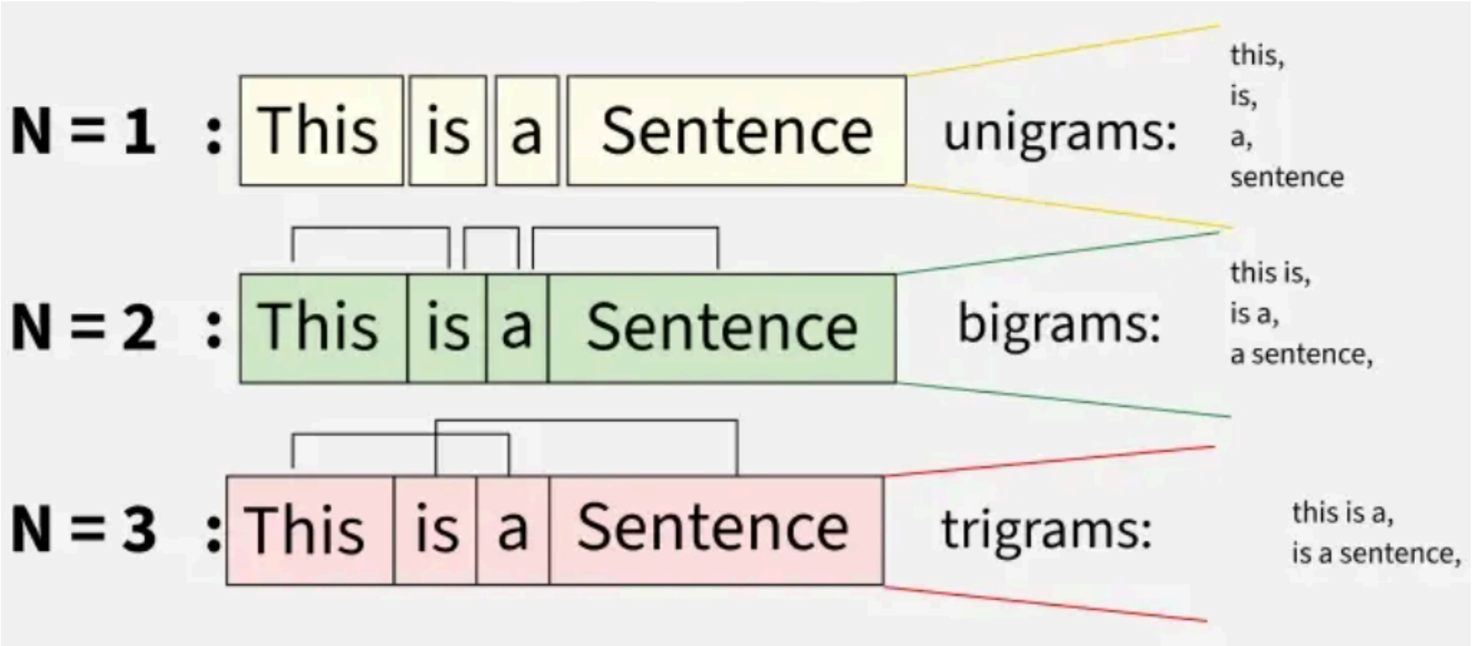


Image Source

Statistical Language Models (N-grams)

Limitations:

- Context length is fixed and short
- Rare word combinations lead to data sparsity
- No understanding of meaning or semantics
- Cannot generalize well to unseen phrases

These limitations restricted their effectiveness on complex language tasks.

Other probabilistic sequence models such as **Hidden Markov Models (HMMs)** were used for tasks like tagging and speech, but were limited in expressiveness.

Neural Language Models: From FeedForward to RNNs

Neural Language Models introduced neural networks into language modeling, allowing models to automatically learn representations from data.
This reduced reliance on manual feature engineering and enabled richer language understanding.

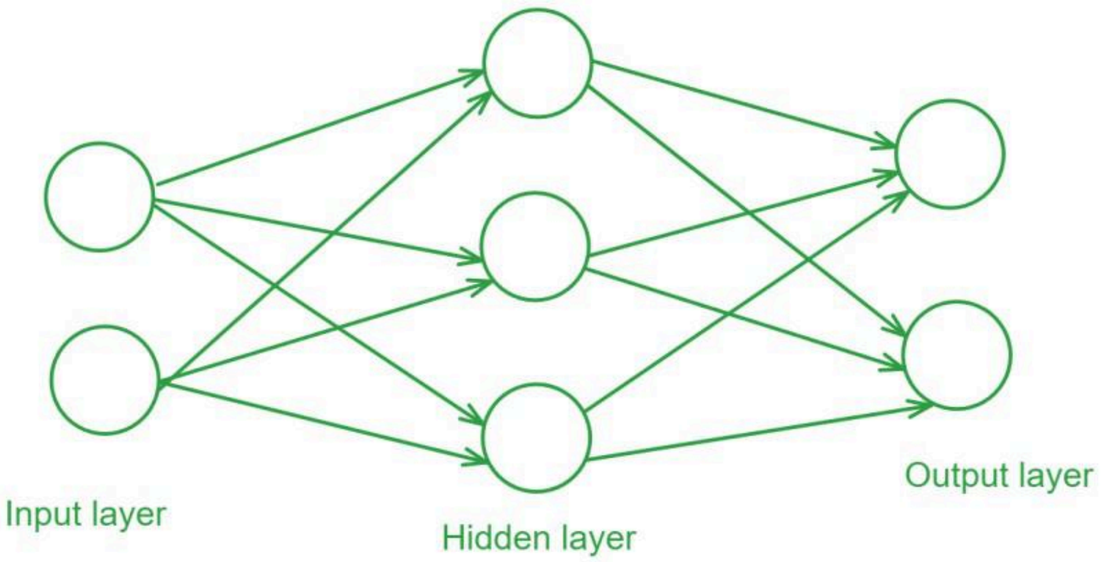


Image Source

Early feedforward neural language models:

- Predicted the next word using a fixed-size context window.
- Used dense word embeddings (Word2Vec) instead of sparse counts.
- Still struggled with long-range dependencies.

These limitations led to sequence-aware neural models.

RNN-Based Models (RNN, LSTM, GRU)

Recurrent Neural Networks (RNNs) were designed specifically to model sequential data like sentences.

They process tokens one at a time while maintaining a hidden state that carries information across time steps, allowing the model to capture word order and context.

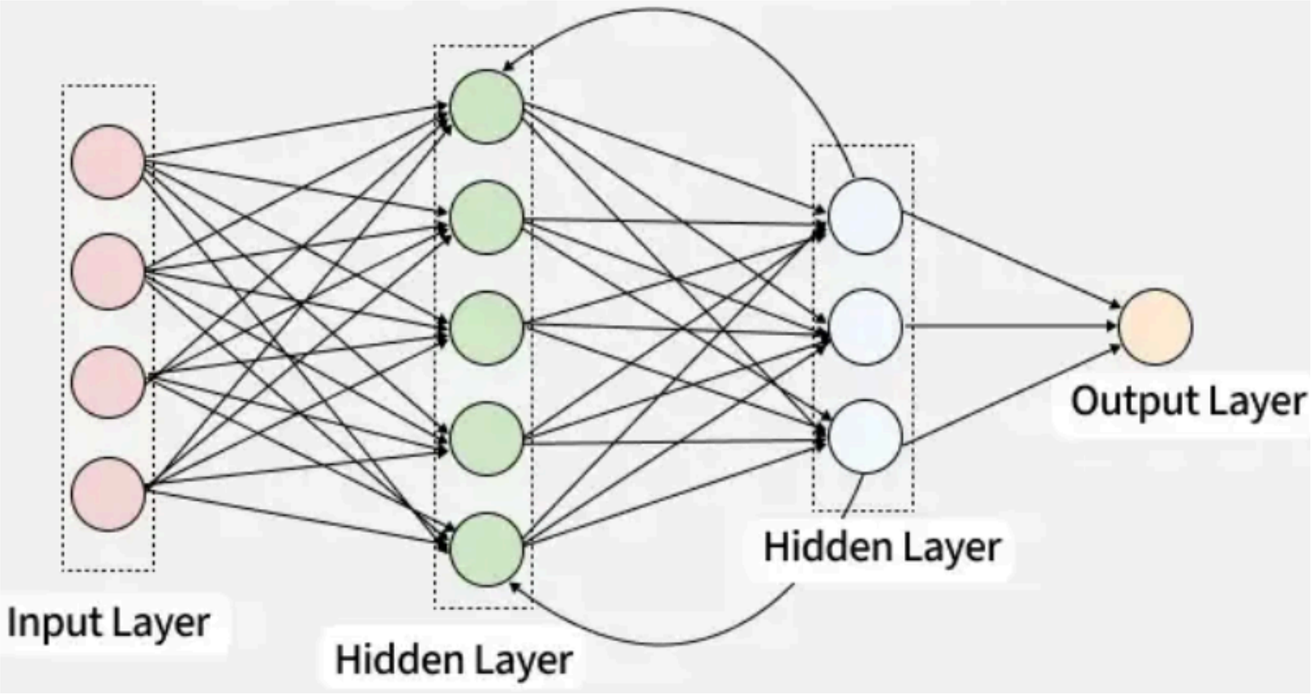


Image Source

RNN-Based Models (RNN, LSTM, GRU)

LSTM (Long Short-Term Memory):

- A specialized RNN designed to remember information over long sequences.
- Uses gates to control what information to store, forget, or output.
- Addresses the vanishing gradient problem.

GRU (Gated Recurrent Unit):

- A simplified version of LSTM
- Uses fewer gates and parameters
- Faster to train while retaining long-context modeling ability

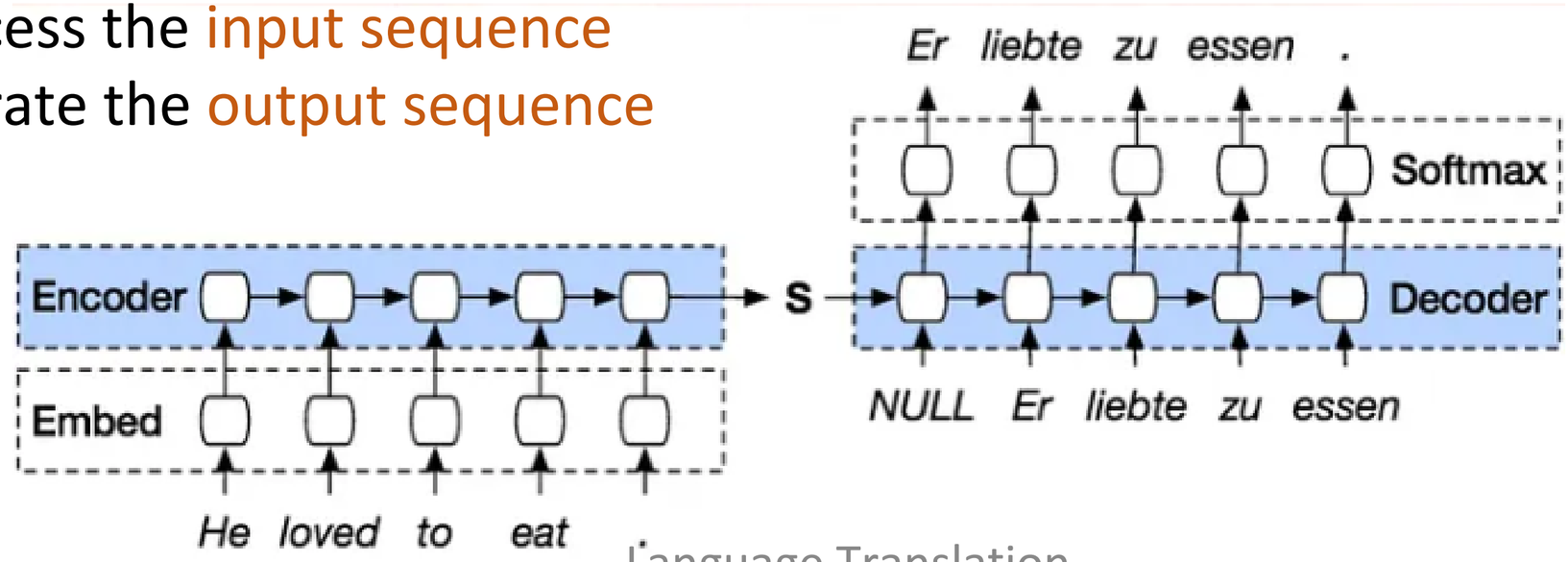
Sequence-to-Sequence Models

Sequence-to-sequence (Seq2Seq) models extended RNN-based language modeling to tasks that transform one sequence into another, such as machine translation and summarization.

They use:

- An encoder RNN to process the input sequence
- A decoder RNN to generate the output sequence

The introduction of attention mechanisms allowed the decoder to dynamically focus on relevant parts of the input sequence, improving handling of long sentences and alignment.



Language Translation

Image Source

Limitations of RNN-Based NLP Models

Despite these advances, RNN-based and Seq2Seq models still faced key challenges:

- Difficulty capturing very long-range dependencies
- Sequential computation leading to slow training and inference
- Limited parallelization on modern hardware (GPUs/TPUs)
- Poor scalability to very large datasets and model sizes

These limitations motivated the development of fully attention-based Transformer architectures.

Transformers

The Transformer architecture, introduced in 2017 in the paper *"Attention Is All You Need"*, revolutionized language modeling.

Transformers replace recurrence with self-attention, allowing models to process entire sequences at once and focus on relevant tokens regardless of distance.

Advantages:

- Strong handling of long-range dependencies.
- Full parallelization during training (inference remains token-by-token).
- Efficient scaling with data, parameters, and compute.

Transformers enabled the modern era of large-scale language models.

Transformer Architecture (High-Level View)

Transformer-based models follow a structured processing pipeline: **Input text → tokens → embeddings.**

Self-attention layers compute relationships between all tokens in the sequence.

Multiple stacked Transformer layers:

- Refine token representations.
- Capture increasingly complex patterns.
- Build deep contextual understanding

This architecture replaced RNNs for large-scale NLP tasks.

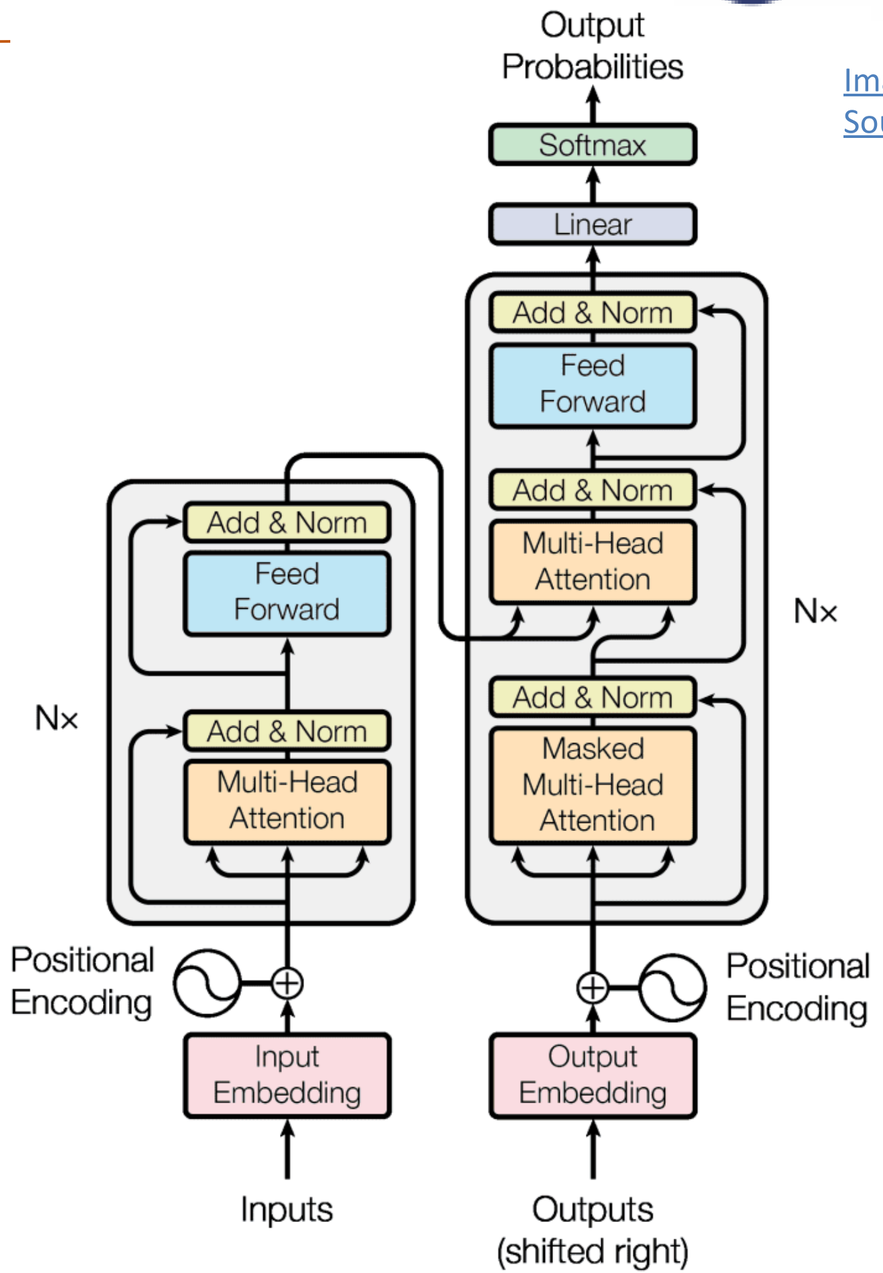


Image Source

Types of Language Models (Encoder/ Decoder/ Encoder-Decoder)

Different Transformer architectures are optimized for different tasks.

Encoder-only models (e.g., BERT):

- Focus on language understanding
- Used for classification, sentiment analysis, and information extraction

Decoder-only models (e.g., GPT):

- Focus on language generation
- Used for chatbots, text completion, and code generation

Encoder–decoder models (e.g., T5, BART):

- Transform one sequence into another
- Used for translation, summarization, and text-to-text tasks

From Transformers to LLMs

Transformers made it possible to train language models at unprecedented scale.

Key enabling factors include:

- Efficient parallel training on modern hardware
- Ability to model long contexts effectively
- Training on massive and diverse datasets

As model size, data, and compute increased, these systems evolved into Large Language Models with strong conversational and reasoning abilities.

Pretrained LLMs (Why Pretraining Works?)

Think of it like learning English before even beginning to write exams.

Stages:

1. Pretraining → general knowledge
2. Fine-tuning → task-specific
3. Alignment → human-friendly

Evolution of Pretrained LLMs

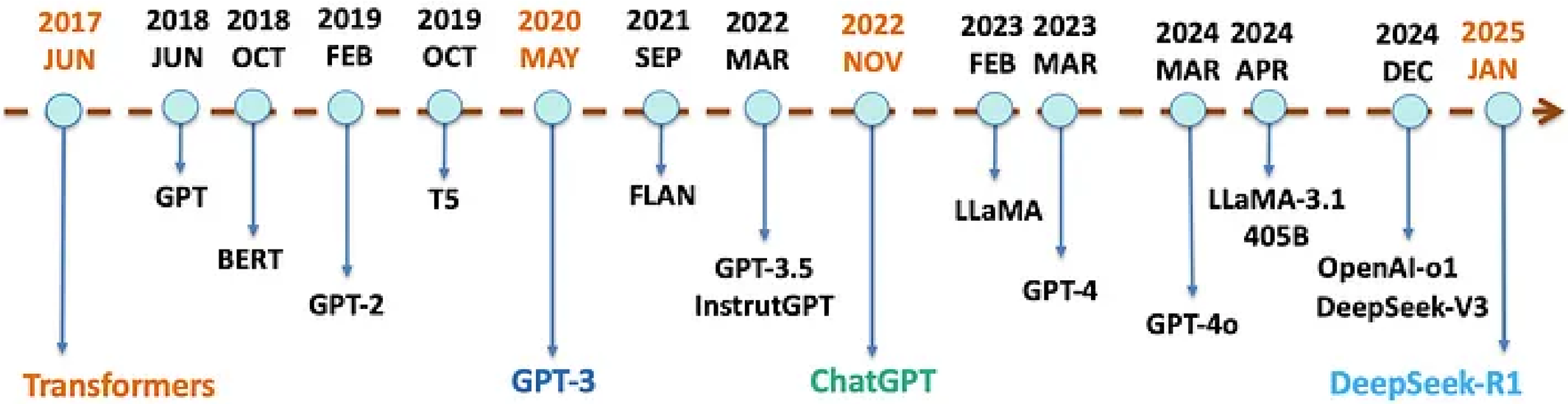


Image Source

Recent Timeline of Major LLM Releases

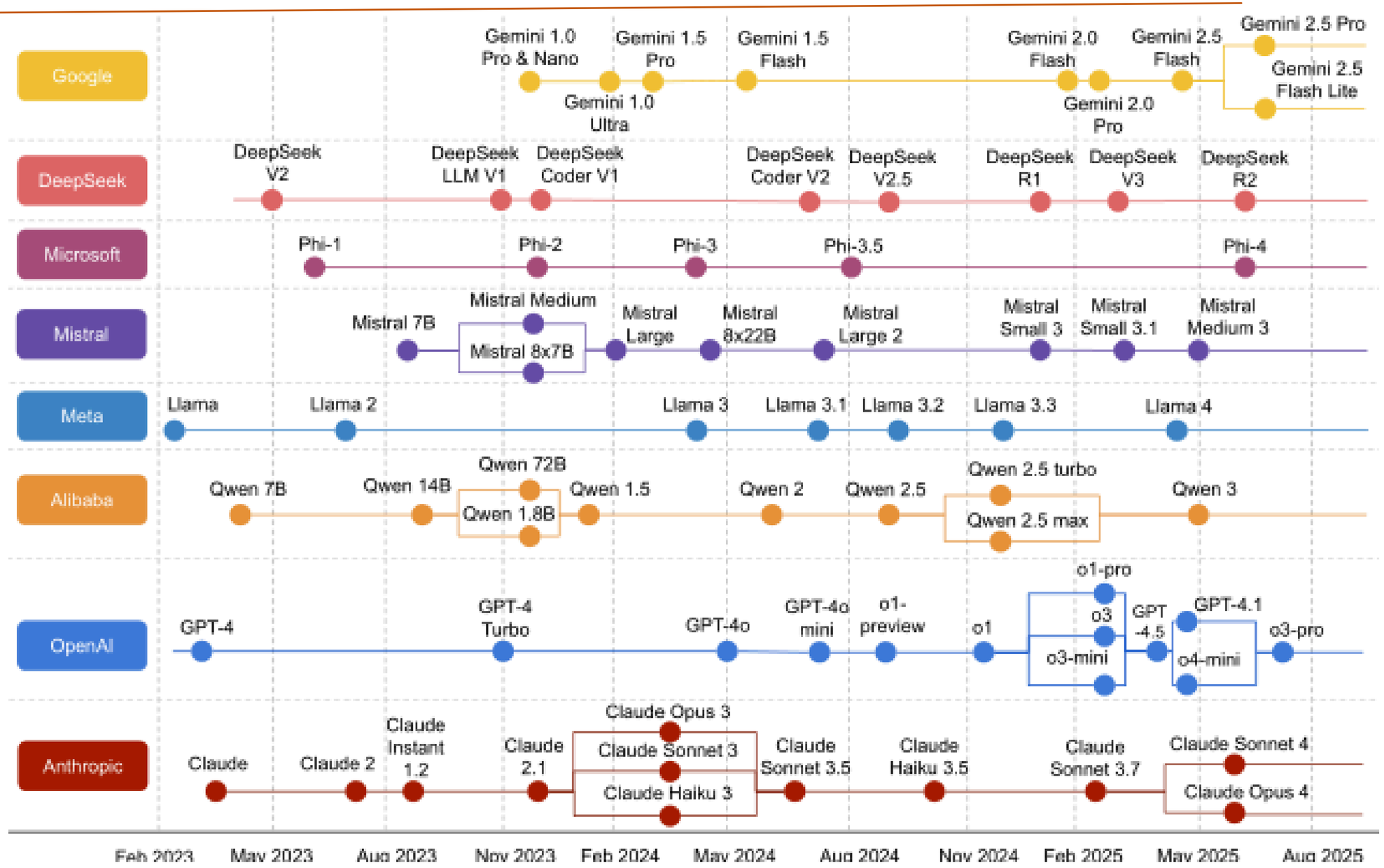


Image Source

Open vs Closed LLMs

Aspect	Open-Source LLMs	Closed / Proprietary LLMs
Access	Model architecture and weights are publicly available	Access limited to API or hosted interface
Transparency	Internals, training details, and behavior can be inspected	Model internals and training data are opaque
Deployment	Can be run locally or on private infrastructure	Cloud-based only, vendor-controlled
Customization	Supports fine-tuning, prompt control, and behavior modification	Limited to prompt-level control
Cost	No license fee, but requires compute resources	Pay-per-use pricing via API
Use cases	Research, learning, experimentation,	Production systems, scalable applications
Examples	LLaMA, Mistral, Falcon	GPT-4, Claude, Gemini

Image, Video & Multimodal GenAI

- Multimodality? Same intelligence, different senses
- Text → LLM
- Image → Vision transformer
- Video → Temporal + vision + language

Why it matters:

- Autonomous driving
- Robotics
- Healthcare

Recent Multimodal Models & Tools to Explore

State-of-the-Art Multimodal Models

- GLM-4.6V : Vision + text + UI understanding, long-context multimodal reasoning
- Falcon 2 VLM: Open-source vision-to-language reasoning
- Molmo 2: Open multimodal model for multi-image and video reasoning
- Emu 3.5: Unified multimodal next-token prediction (research, open code)

Video Generation Models

- LTX-2: Open-source video + audio generation

Sandboxes and other stuff to explore

Interactive / Playground Tools

- Runway Gen-4: Creative sandbox for text-to-video generation
- https://www.nvidia.com/en-us/research/ai-playground/
- https://huggingface.co/learn/llm-course/chapter0/1
- https://aiengineering.academy/

Few resources fyi

1. Retrieval-Augmented Generation (RAG)
   - Curated resources: Awesome RAG
   - End-to-end RAG system: RAGFlow
   - Advanced retrieval + reasoning: R2R

2. Agentic AI (LLMs that plan, call tools, and execute multi-step tasks)
   - Agent frameworks & projects: https://github.com/topics/agentic-ai
   - Real LLM applications (agents + RAG)
   - Multi-agent RAG tutorial (hands-on)

Few resources fyi

3. Smaller & Efficient Models (Focus on efficiency over sheer scale)
   - Lightweight RAG research framework: MiniRAG

4. On-Device / Offline LLMs (Low latency, privacy-friendly, works without internet)
   - Local LLM runner & agent framework: AMD Gaia

Why this matters:
- Phone AI that works offline
- Faster response, better privacy
- Real-world deployment on edge devices

MCQs

1. Why did rule-based NLP systems fail to scale effectively?

A. They lacked sufficient computing power
B. They required excessive labeled data
C. Language rules grow combinatorially with complexity
D. They could not tokenize text

Why: Handling ambiguity and variation requires exponentially many handcrafted rules.

MCQs

2. What is a key limitation of n-gram language models?

A. They cannot be trained on large datasets
B. They capture semantics too strongly
C. They rely on fixed-length context windows
D. They require neural networks

Why: N-gram models only condition on the previous n−1 tokens, limiting context understanding.

MCQs

3. Why were word embeddings a major breakthrough over one-hot encodings?

A. They reduce vocabulary size

B. They capture semantic similarity between words

C. They eliminate sparsity entirely

D. They enforce grammatical correctness

Why: Embeddings place semantically related words closer in vector space, enabling meaning-aware learning.

MCQs

4. What problem did LSTM and GRU architectures attempt to solve?

A. Lack of parallel computation
B. Tokenization inefficiency
C. Vanishing gradients in long sequences
D. Overfitting in small datasets

Why: Gating mechanisms help preserve information over longer sequences by controlling gradient flow.

MCQs

5. Why was the attention mechanism a turning point in NLP?

A. It removed the need for embeddings

B. It allowed models to dynamically focus on relevant context

C. It replaced probability-based learning

D. It eliminated training data requirements

Why: Attention enables selective focus on important tokens, improving long-range dependency modeling.

MCQs

6. What distinguishes Transformers from RNN-based architectures?

A. Use of convolution instead of recurrence
B. Sequential processing of tokens
C. Full parallelization using self-attention
D. Dependence on handcrafted features

Why: Transformers process all tokens simultaneously using self-attention, enabling efficient scaling.

Sample 4 markers

1. Compare rule-based NLP systems and statistical language models, highlighting their limitations.
2. Explain why word embeddings were critical for the development of neural language models.
3. Describe how attention mechanisms address the shortcomings of RNN-based models.
4. Explain why Transformers enabled the emergence of modern Large Language Models.

References to look into further

1. Jurafsky & Martin — Speech and Language Processing (Selected Chapters)
2. Jay Alammar — The Illustrated Transformer
3. Mikolov et al. (2013) — Efficient Estimation of Word Representations in Vector Space
4. Hochreiter & Schmidhuber (1997) — Long Short-Term Memory
5. Devlin et al. (2018) — BERT: Pre-training of Deep Bidirectional Transformers

THANK YOU

Dr. Uma D

Professor, Dept. of CSE

umaprabha@pes.edu